

Modulo Didattico 2 – Costruire ed importare dati esterni, funzioni per la sintesi di strutture tabellari

Corso Analisi dei dati con applicazioni informatiche

Ottavia M. Epifania
University of Trento
`ottavia.epifania@unitn.it`

A.A. 2025/2026

- 1 File esterni di dati
- 2 Variabili
- 3 Statistiche descrittive di base
- 4 Generazione di variabili casuali e grafici

1 File esterni di dati

- Working Directories (wd)
- Caricare
- Sintesi di strutture tabellari

2 Variabili

3 Statistiche descrittive di base

4 Generazione di variabili casuali e grafici

`getwd()`

①

- ① Restituisce la wd in cui ci si trova (default: cartella dove si trova l'installazione di R)

```
[1] "C:/Users/Ottavia/Documents/GitHub/stpc-analisi-dati/slides/modulo2"
```

`setwd("C/Documenti/I-miei-file")`

②

- ② Permette di settare una nuova wd

`dir()`

③

- ③ Vettore che indica tutti i file/sottocartelle presenti nella wd ottenuta con `getwd()`

```
[1] "img" "Kernel_esempio_di_calcolo.pdf"
[3] "Laboratorio" "Modulo2.pdf"
[5] "Modulo2.qmd" "Modulo2.rmarkdown"
```

Esercizio

- ① Aprire un nuovo script in Rstudio
 - File, New File, R script oppure tasti `shift + ctrl + n`
- ② Determinare in che wd ci si trova (copia & incolla il risultato del comando `getwd()` nello script sottoforma di commento)
- ③ Creare una nuova cartella sul desktop dal nome `corso-analisi`
- ④ Usare il comando `setwd()` per spostarsi in quella cartella

1 File esterni di dati

- Working Directories (wd)
- Caricare
- Sintesi di strutture tabellari

2 Variabili

3 Statistiche descrittive di base

4 Generazione di variabili casuali e grafici

```
read.csv2("file_path")
```

Se si possiede un dataset chiamato "Dati" nella cartella "EXP", questo codice lo importerà all'interno dell'ambiente

```
# Lettura file dati esterni
```

```
DATI <- read.csv2("C:\\EXP\\Dati.csv")
```

1 File esterni di dati

- Working Directories (wd)
- Caricare
- Sintesi di strutture tabellari

2 Variabili

3 Statistiche descrittive di base

4 Generazione di variabili casuali e grafici

summary()

summary(data)

Riassume il contenuto di strutture di dati

summary(rock)

area	peri	shape	perm
Min. : 1016	Min. : 308.6	Min. : 0.09033	Min. : 6.30
1st Qu.: 5305	1st Qu.: 1414.9	1st Qu.: 0.16226	1st Qu.: 76.45
Median : 7487	Median : 2536.2	Median : 0.19886	Median : 130.50
Mean : 7188	Mean : 2682.2	Mean : 0.21811	Mean : 415.45
3rd Qu.: 8870	3rd Qu.: 3989.5	3rd Qu.: 0.26267	3rd Qu.: 777.50
Max. : 12212	Max. : 4864.2	Max. : 0.46413	Max. : 1300.00

str()

```
str(data)
```

Fornisce informazioni circa la struttura generale di un oggetto:

str()

```
str(data)
```

Fornisce informazioni circa la struttura generale di un oggetto:

```
str(rock)
```

```
'data.frame':   48 obs. of  4 variables:
 $ area : int   4990 7002 7558 7352 7943 7979 9333 8209 8393 6425
 $ peri : num   2792 3893 3931 3869 3949 ...
 $ shape: num   0.0903 0.1486 0.1833 0.1171 0.1224 ...
 $ perm : num    6.3  6.3  6.3  6.3 17.1 17.1 17.1 17.1 119 119 ...
```

head() (α) and tail() (ω)

head()

Le prime 6 osservazioni del dataset

head(rock)

	area	peri	shape	perm
1	4990	2791.90	0.0903296	6.3
2	7002	3892.60	0.1486220	6.3
3	7558	3930.66	0.1833120	6.3
4	7352	3869.32	0.1170630	6.3
5	7943	3948.54	0.1224170	17.1
6	7979	4010.15	0.1670450	17.1

tail()

le ultime 6 osservazioni del dataset

tail(rock)

	area	peri	shape	perm
43	5605	1145.690	0.464125	1300
44	8793	2280.490	0.420477	1300
45	3475	1174.110	0.200744	580
46	1651	597.808	0.262651	580
47	5514	1455.880	0.182453	580
48	9718	1485.580	0.200447	580

apply()

`apply(data, MARGIN, FUN)`

data: i dati su cui si vuole lavorare

MARGIN: 1 applica la funzione sulle righe (attraverso le colonne), 2 applica la funzione sulle colonne (attraverso le righe)

FUN: la funzione che si vuole applicare

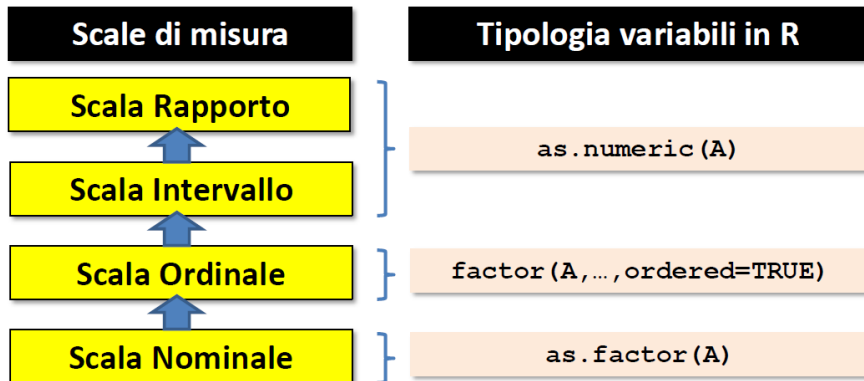
```
apply(rock, 1, sum)  # Somma per riga
```

```
[1] 7788.290 10901.049 11495.143 11227.737 11908.762 12006.417 13696.040
[8] 12571.014 12194.244  9642.812 13963.201 12729.388 14770.169 12468.672
[15] 13498.943 12585.623 15629.464 15589.483 16801.020 14405.155 11408.854
[22] 16371.431 17051.890 11893.602  9077.671  6312.529 10072.432  7571.553
[29]  9250.435  7025.001  9393.452 10245.336  8356.958  7258.715  9692.914
[36] 10305.336  4945.877  2044.761  4813.624  7012.214  7289.872  2624.872
[43]  8051.154 12373.910  5229.311  2829.071  7550.062 11783.780
```

```
apply(rock, 2, mean)  # Media per colonna
```

```
      area      peri      shape      perm
7187.7291667 2682.2119375  0.2181104 415.4500000
```

- 1 File esterni di dati
- 2 Variabili**
- 3 Statistiche descrittive di base
- 4 Generazione di variabili casuali e grafici



- 1 File esterni di dati
- 2 Variabili
 - Categoricali - Nominali
 - Categoricali - Ordinali
- 3 Statistiche descrittive di base
- 4 Generazione di variabili casuali e grafici

as.factor()

as.factor(): Converte una variabile in una variabile di tipo **categoriale/nominale** con tanti livelli quanti sono i valori unici nel vettore

Due livelli

```
Genere <- c("m", "f", "f", "m", "f", "m")
```

Genere

```
[1] "m" "f" "f" "m" "f" "m"
```

as.factor()

`as.factor()`: Converte una variabile in una variabile di tipo **categoriale/nominale** con tanti livelli quanti sono i valori unici nel vettore

Due livelli

```
Genere <- c("m", "f", "f", "m", "f", "m")
```

Genere

```
[1] "m" "f" "f" "m" "f" "m"
```

```
as.factor(Genere)
```

```
[1] m f f m f m
```

```
Levels: f m
```

Più livelli

```
diagnosi <- c("schizo", "dep", "narc", "schizo", "dep", "narc")  
diagnosi
```

```
[1] "schizo" "dep"    "narc"   "schizo" "dep"    "narc"
```

Più livelli

```
diagnosi <- c("schizo", "dep", "narc", "schizo", "dep", "narc")  
diagnosi
```

```
[1] "schizo" "dep"      "narc"     "schizo" "dep"      "narc"
```

```
as.factor(diagnosi)
```

```
[1] schizo dep      narc      schizo dep      narc  
Levels: dep narc schizo
```

- 1 File esterni di dati
- 2 Variabili
 - Categoricali - Nominali
 - Categoricali - Ordinali
- 3 Statistiche descrittive di base
- 4 Generazione di variabili casuali e grafici

```
factor(A,levels=c("a1","a2",..., "ak"),  
       ordered=TRUE)
```

Converte la variabile in una variabile con tanti livelli quanti sono i valori unici e viene forzato un ordine su questi livelli:

```
Voto <- c("alto","medio","medio","basso","alto")  
Voto
```

```
[1] "alto" "medio" "medio" "basso" "alto"
```

```
factor(Voto,levels=c("basso","medio","alto"), ordered=TRUE)
```

```
[1] alto medio medio basso alto  
Levels: basso < medio < alto
```

- 1 File esterni di dati
- 2 Variabili
- 3 Statistiche descrittive di base**
- 4 Generazione di variabili casuali e grafici

- 1 File esterni di dati
- 2 Variabili
- 3 Statistiche descrittive di base**
 - Tavole di frequenza
- 4 Generazione di variabili casuali e grafici

```
table(variabile)
```

Calcola le occorrenze (frequenze) per ogni categoria. Funziona sia per le variabili categoriali sia per le variabili numeriche

```
table(Genere)
```

```
Genere
```

```
f m
```

```
3 3
```

```
table(diagnosi)
```

```
diagnosi
```

```
dep    narc  schizo
```

```
2      2      2
```

Altre funzioni

Funzione	Significato
<code>mean()</code>	Media
<code>min(), max()</code>	Minimo, Massimo
<code>range()</code>	Range di valori entro cui varia la variabile
<code>quantile(x, probs)</code>	Calcola il quantile $p=\text{probs}$ della variabile numerica X
<code>sd(), var()</code>	Deviazione st., Varianza
<code>sum()</code>	Somma

- 1 File esterni di dati
- 2 Variabili
- 3 Statistiche descrittive di base
- 4 Generazione di variabili casuali e grafici**

- 1 File esterni di dati
- 2 Variabili
- 3 Statistiche descrittive di base
- 4 Generazione di variabili casuali e grafici
 - Generazioni di variabili
 - Istogrammi
 - Boxplot
 - Boxplot – Calcolo degli outlier
 - Istogramma con kernel
 - Utilizzare i kernel

set.seed()

Permette di settare un seed comune a tutt

Garantisce la perfetta replicabilità dei risultati

Garantisce che se tutt settiamo questo seed, otteniamo gli stessi grafici

```
set.seed(999)
```

`rnorm(n, mean = 0, sd = 1)` :Genera n valori casuali e indipendenti da una variabile casuale normale con media `mean` 0 e deviazione standard `sd` 1 (normale standard)

`rnorm(n, mean = 0, sd = 1)` :Genera n valori casuali e indipendenti da una variabile casuale normale con media `mean` 0 e deviazione standard `sd` 1 (normale standard)

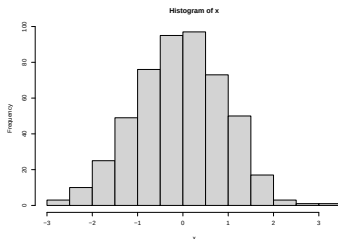
`runif(n, min = 0, max = 1)` Genera n valori casuali e indipendenti da una variabile casuale uniforme, dove i valori sono compresi tra 0 (`min`) e 1 (`max`)

Generazioni di variabili

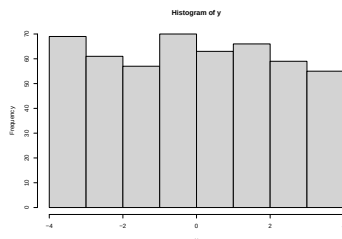
`rnorm(n, mean = 0, sd = 1)` :Genera n valori casuali e indipendenti da una variabile casuale normale con media `mean` 0 e deviazione standard `sd` 1 (normale standard)

`runif(n, min = 0, max = 1)` Genera n valori casuali e indipendenti da una variabile casuale uniforme, dove i valori sono compresi tra 0 (`min`) e 1 (`max`)

`x = rnorm(500, mean = 0, sd = 1)`



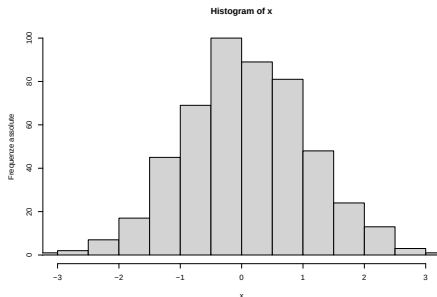
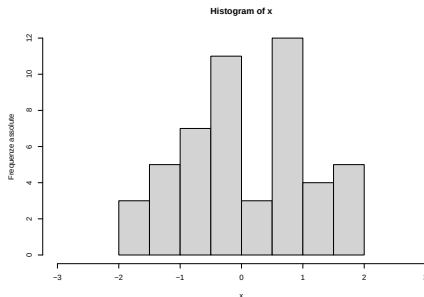
`y = runif(500, min = -4, max = 4)`



- 1 File esterni di dati
- 2 Variabili
- 3 Statistiche descrittive di base
- 4 Generazione di variabili casuali e grafici
 - Generazioni di variabili
 - Istogrammi
 - Boxplot
 - Boxplot – Calcolo degli outlier
 - Istogramma con kernel
 - Utilizzare i kernel

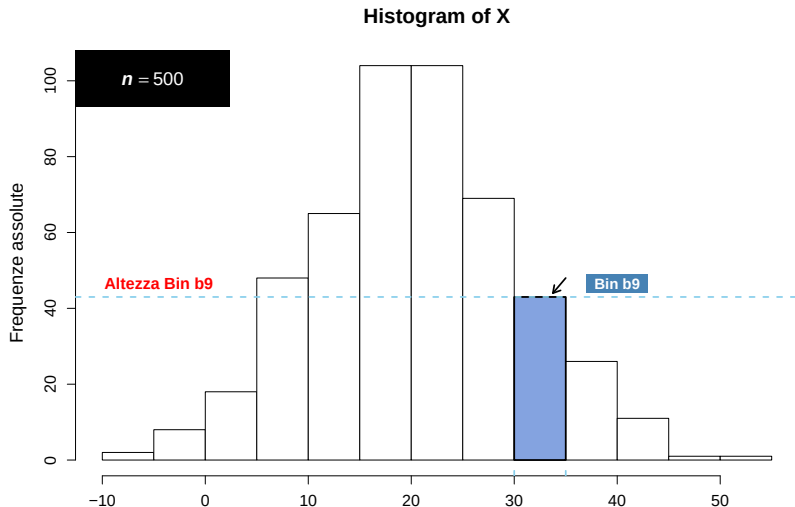
Istogrammi

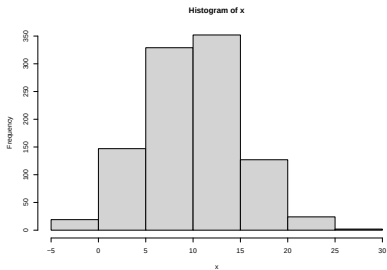
`hist(x, breaks)` Produce un istogramma a partire da un vettore `x` di osservazioni quantitative. `breaks` definisce la larghezza delle singole barre, secondo diverse regole

(a) $n = 500$ (a) $n = 50$

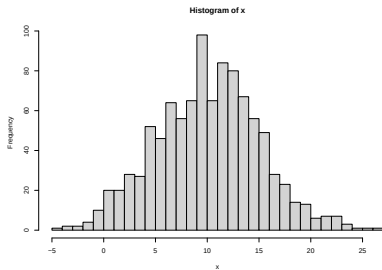
Istogrammi con n diverso

Istogrammi

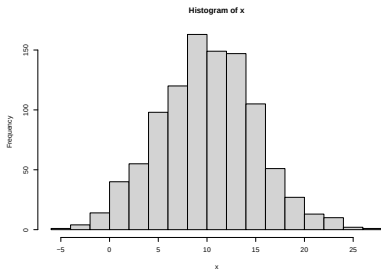




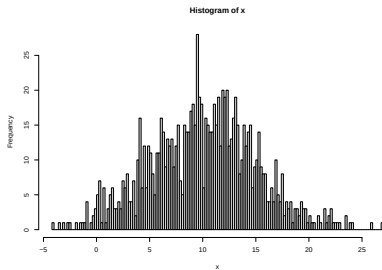
(a) Sturges - Default



(a) FD



(a) Scott



(a) User defined (145)

Sturges:

$$\frac{\max(x) - \min(x)}{\log_2(n + 1)}$$

Scott:

$$3.5 \sqrt{\text{var}(x)} n^{-1/3}$$

Freedman-Diaconis

$$2 (Q_3 - Q_1) n^{-1/3}$$

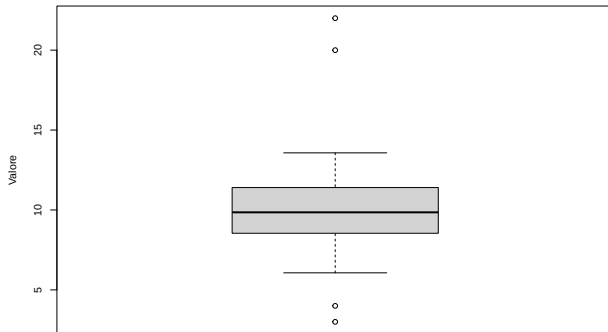
User defined:

$$\frac{\max(x) - \min(x)}{\text{breaks}}$$

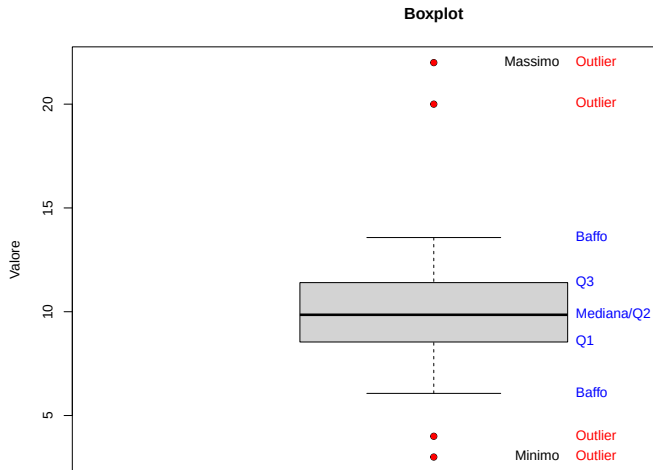
- 1 File esterni di dati
- 2 Variabili
- 3 Statistiche descrittive di base
- 4 Generazione di variabili casuali e grafici
 - Generazioni di variabili
 - Istogrammi
 - Boxplot
 - Boxplot – Calcolo degli outlier
 - Istogramma con kernel
 - Utilizzare i kernel

Boxplot

`boxplot(x)` Produce un boxplot a partire da un vettore X di osservazioni quantitative



Boxplot



- 1 File esterni di dati
- 2 Variabili
- 3 Statistiche descrittive di base
- 4 Generazione di variabili casuali e grafici
 - Generazioni di variabili
 - Istogrammi
 - Boxplot
 - Boxplot – Calcolo degli outlier
 - Istogramma con kernel
 - Utilizzare i kernel

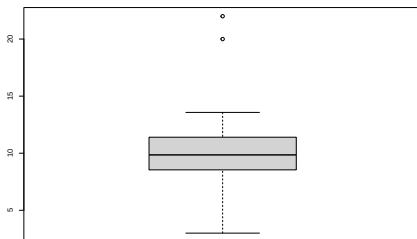
La lunghezza dei baffi (e di conseguenza la definizione degli outliers) è definita attraverso il metodo Tukey:

$$[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR]$$

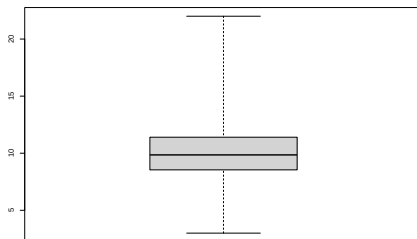
dove $IQR = Q_3 - Q_1$ (Inter-quartile range)

Settando l'argomento `range = x` si può cambiare il modo in cui vengono calcolati i baffi:

```
boxplot(x, range = 2)
boxplot(x, range = 0)
```



(a) range = 2 - Meno Conservativo

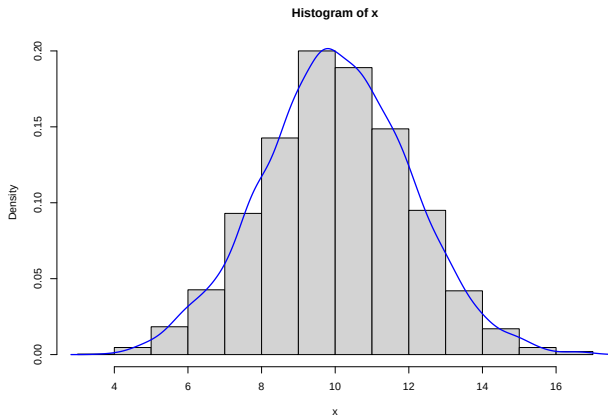


(a) range = 0 - Minimo e Massimo

- 1 File esterni di dati
- 2 Variabili
- 3 Statistiche descrittive di base
- 4 Generazione di variabili casuali e grafici
 - Generazioni di variabili
 - Istogrammi
 - Boxplot
 - Boxplot – Calcolo degli outlier
 - Istogramma con kernel
 - Utilizzare i kernel

Istogramma con kernel

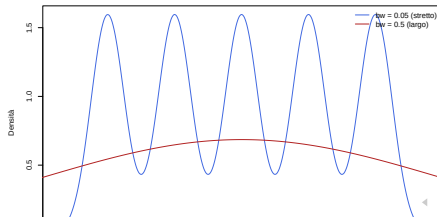
kernel density estimation, KDE: Metodo non parametrico per *stimare* la funzione di densità di probabilità di una variabile casuale continua a partire da un campione di dati



Funzione che “assegna” una probabilità attorno a ogni singolo dato osservato

`density(X, bw)`: Stima della densità di una variabile a partire da un vettore `X` di osservazioni quantitative. `bw`: Larghezza della banda con cui viene stimato il kernel attorno a ogni dato:

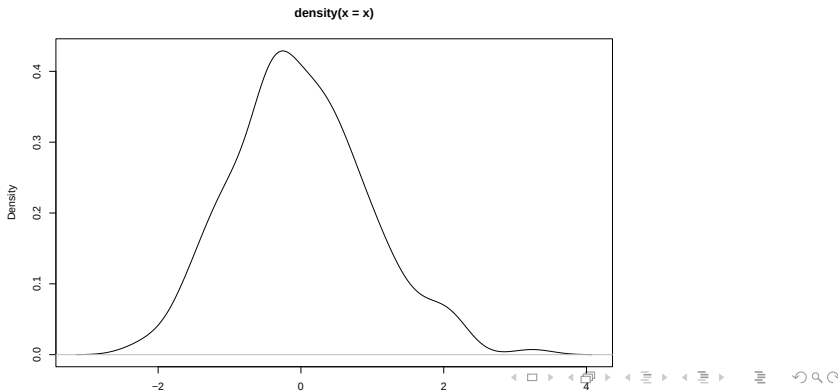
- Più è stretto attorno al dato: Si vede bene la struttura del dato, ma si rischia di essere in overfitting (quella stima va bene SOLO per quel dato)
- Più è largo attorno al dato: La curva è ben definita e liscia, ma si rischia di perdere dettagli per questi specifici dati (underfitting)



- 1 File esterni di dati
- 2 Variabili
- 3 Statistiche descrittive di base
- 4 Generazione di variabili casuali e grafici
 - Generazioni di variabili
 - Istogrammi
 - Boxplot
 - Boxplot – Calcolo degli outlier
 - Istogramma con kernel
 - Utilizzare i kernel

La funzione calcola la densità, per vedere la densità va usata insieme alla funzione `plot()`

```
x = rnorm(200)  
plot(density(x))
```



va imposto all'istogramma che costruisca i bin con la densità e non con la frequenza:

```
hist(x, freq = FALSE)
lines(density(x), lwd = 2, col = "blue")
```

